DreamCoder Run Scripts: Architecture Documentation

This document provides comprehensive documentation of all run scripts, their architectures, enumeration strategies, and neural guidance flows.

Table of Contents

Executive Summary

The codebase contains multiple run scripts that evolved over time, each with different optimization strategies and architecture designs. The key distinction is:

Script Family	Enumerator	Neural Guidance	Parallelization
`run_full_dreamcoder.py`	`enumerate_simple`	☐ Not used	Sequential
`run_overnight_optimized.py`	`enumerate_simple`	☐ Not used	PyPy workers
`run_overnight_cython.py`	`enumerate_simple`	☐ Trained but NOT applied	PyPy workers
`run_overnight_v3.py`	`enumerate_simple`	☐ Trained but NOT applied	PyPy workers (via cython)
`run_factorial_experiment.py`	`TempDownEnumerator`	☐ Properly applied	CPython workers
`run_overnight_set_transformer.py`	`enumerate_simple`	☐ Trained but NOT applied	PyPy workers

Key Finding: Most overnight scripts train a recognition model but **never actually use** its predictions to guide enumeration in workers.

Enumeration Strategies

1. `enumerate_simple` (Iterative Deepening)

Algorithm:

```
for depth in 1, 2, 3, ..., max_depth:
    for each program P of exactly depth d:
        if type(P) matches request_type:
            yield P
```

Characteristics: - Enumerates ALL programs at depth d before ANY at depth d+1 - Guarantees shortest programs found first - **Does NOT use grammar probabilities** for ordering - Simple to implement but inefficient

Example enumeration order (depth 1 □ 2 □ 3):

```
Depth 1: true, false, 0, 1, ...
Depth 2: (not true), (eq 0 1), (+ 1 1), ...
Depth 3: (and true (not false)), ...
```

2. `TopDownEnumerator` **(Best-First Hole-Filling)**

Algorithm:

```
priority_queue = [(cost=0, program=?:request_type)]

while priority_queue not empty:
    cost, partial_program = pop_min(priority_queue)

    if no holes in partial_program:
        yield partial_program  # Complete!
        continue

    hole = find_first_hole(partial_program)
    for production in grammar.productions:
        if production.type unifies with hole.type:
            new_program = fill_hole(partial_program, hole, production)
            new_cost = cost + (-log_prob(production))
            push(priority_queue, (new_cost, new_program))
```

Characteristics: - **Uses grammar probabilities** for ordering (best-first search) - Programs enumerated in order of **increasing cost** (decreasing probability) - Memory efficient: only stores partial programs - Can integrate neural guidance by modifying grammar weights

Example enumeration with probabilities:

```
p(eq)=0.1, p(all_same_suit)=0.2, p(true)=0.05

Priority queue evolution:
1. [(?:hand→bool, cost=0)]
2. [( .?:bool, cost=0)]  -- lambda is free
3. [( .true, cost=2.99), ( .(eq ?:int ?:int), cost=2.30), ( .(all_same_suit ?:hand), cost=1.61)]
   ↑ all_same_suit is highest probability → enumerated first
```

Run Script Architectures

Architecture 1: `run_full_dreamcoder.py`

Purpose: Original DreamCoder experiment runner with full wake-sleep components.

Schema:

```
                MAIN PROCESS (CPython)


                FullDreamCoder.run()

    for iteration in 1..max_iterations:


      WAKE PHASE

      for task in tasks:
          enumerate_simple(grammar, task.request_type)
          # Sequential, no neural guidance

                          ↓

      SLEEP - COMPRESSION

      compress_frontiers(grammar, solved_frontiers)
      → new_grammar with Invented abstractions

                          ↓

      SLEEP - RECOGNITION (optional)

      recognition.train_on_frontiers(solved_tasks)
      # Trains but predictions NOT USED in next iteration
```

Key Files: - `dreamcoder_core/full_dreamcoder.py` - Main DreamCoder class - `dreamcoder_core/enumeration` - `enumerate_simple` function

Enumerator: `enumerate_simple` (iterative deepening) **Neural Guidance:** Trained but NOT applied during enumeration **Parallelization:** Sequential (single-threaded)

Architecture 2: `run_overnight_optimized.py` **/** `run_overnight_cython.py`

Purpose: Optimized overnight training with PyPy workers for parallel enumeration.

Schema:

```
                    MAIN PROCESS (CPython + optional Cython)


                CythonOptimizedDreamCoder._run_iteration()


      WAKE PHASE (PARALLEL)

       ProcessPoolExecutor(max_workers=4)

              spawn PyPy worker
              spawn PyPy worker
              spawn PyPy worker      JSON serialization
              spawn PyPy worker       of task data ONLY
                                          (no grammar weights!)


    Recognition model exists but predict_grammar_weights() NOT called
```

```
      PyPy WORKER PROCESS 1                  PyPy WORKER PROCESS 2


  # enumeration_worker.py                  # enumeration_worker.py

  grammar = build_lean_grammar()           grammar = build_lean_grammar()
  # FRESH DEFAULT GRAMMAR!                 # FRESH DEFAULT GRAMMAR!

  if grammar_productions:                  if grammar_productions:
      pass  # NEVER EXECUTED!                  pass  # NEVER EXECUTED!

  enumerate_simple(grammar, ...)           enumerate_simple(grammar, ...)

  return JSON results                      return JSON results
```

Critical Issue in `enumeration_worker.py`:

```
# Line 54: Always builds fresh default grammar
grammar = build_lean_grammar()
```

```
# Lines 57-59: Placeholder that NEVER executes
if grammar_productions:
    pass  # grammar_productions is always None!
```

Critical Issue in `run_pypy_worker()`:

```
input_data = {
    'task': task_data,
    'grammar_productions': None,  # HARDCODED TO NONE!
    ...
}
```

Enumerator: `enumerate_simple` (in PyPy workers) **Neural Guidance:** Trained in main process, **NEVER** sent to workers **Parallelization:** PyPy subprocess workers (3-6x speedup)

Architecture 3: `run_overnight_v3.py`

Purpose: "Airtight Edition" - wraps `run_overnight_cython.py` with validation.

Schema:

```
                    run_overnight_v3.py


    PreFlightValidator
      - validate_imports()
      - validate_grammar()
      - validate_rules()
      - validate_task_creation()
      - validate_recognition_model()
      - validate_pypy_worker()
      - validate_scrambling_fix()




    PhaseConfig × 4
      Phase 1: Easy rules, budget=150k, depth=7
      Phase 2: All rules, budget=250k, depth=8
      Phase 3: Deep search, budget=400k, depth=9
      Phase 4: Intensive, budget=600k, depth=10




    CythonOptimizedDreamCoder.run()
```

```
(from run_overnight_cython.py)
- Same architecture as Architecture 2
- Same neural guidance issue
```

Enumerator: `enumerate_simple` (via `run_overnight_cython.py`) **Neural Guidance:** Same issue as Architecture 2 **Parallelization:** PyPy subprocess workers

---

Architecture 4: `run_factorial_experiment.py` □

Purpose: 2×3×3 factorial experiment with proper neural guidance.

Schema:

```
                MAIN PROCESS (CPython)


                  run_single_condition()

for iteration in 1..max_iterations:


  WAKE PHASE (PARALLEL with neural guidance)

   for task in tasks:
       predicted_log_probs = recognition.predict_grammar_weights()
       # ↑ Neural guidance extracted BEFORE spawning workers

   ProcessPoolExecutor(max_workers=6)

          _enumerate_task_worker() with predicted_log_probs
            # Weights serialized as dict {prim_name: log_prob}




                WORKER PROCESS (CPython)
```

```
def _enumerate_task_worker(args):
    predicted_log_probs = args.get('predicted_log_probs')  # From main!
    blend_factor = args.get('blend_factor', 0.5)

    grammar = build_grammar_for_variant(...)

    # APPLY neural guidance to grammar
    if predicted_log_probs is not None:
        for prod in grammar.productions:
            prim_name = str(prod.program)
            if prim_name in predicted_log_probs:
                new_lp = (1-blend) * prod.log_prob + blend * predicted[...]
        grammar = Grammar(new_productions).normalize_probabilities()

    # Use TopDownEnumerator (best-first with neural-guided grammar)
    enumerator = TopDownEnumerator(grammar, ...)
    for program, log_prob in enumerator.enumerate(...):
        ...
```

Key Implementation (lines 117-138):

```python
# Apply recognition model predictions to grammar weights
if predicted_log_probs is not None:
    new_productions = []
    for prod in grammar.productions:
        prim_name = str(prod.program)
        if prim_name in predicted_log_probs:
            # Blend original and predicted with configurable factor
            new_lp = (1 - blend_factor) * prod.log_probability + \
                     blend_factor * predicted_log_probs[prim_name]
        else:
            new_lp = prod.log_probability
        new_productions.append(Production(prod.program, prod.tp, new_lp))

    grammar = Grammar(new_productions, grammar.log_variable).normalize_probabilities()
```

Enumerator: `TopDownEnumerator` (best-first, uses grammar probabilities) **Neural Guidance:** ✓ Properly applied via `predicted_log_probs` dict **Parallelization:** CPython ProcessPoolExecutor (can't use PyPy with TopDownEnumerator)

Architecture 5: `run_overnight_set_transformer.py`

Purpose: Test Set Transformer recognition model architecture.

Schema: Same as Architecture 2, but with: - `SetTransformerRecognitionModel` instead of `NeuralRecognitionModel` - Hierarchical encoding: cards ✓ hand ✓ task - Position-aware

attention for positional rules

Enumerator: `enumerate_simple` (PyPy workers) **Neural Guidance:** Trained but NOT applied (same issue as Architecture 2) **Parallelization:** PyPy subprocess workers

---

Neural Guidance Flow

How Neural Guidance SHOULD Work

```
                    DREAMCODER NEURAL GUIDANCE FLOW


   1. TRAIN RECOGNITION MODEL (Sleep Phase)

      Input: (task_examples, solution_program) pairs
      Output: model that predicts useful primitives

   2. PREDICT GRAMMAR WEIGHTS (Before Wake Phase)

      guided_grammar = recognition.predict_grammar_weights(task)

      This creates a NEW grammar where:
      - Primitives predicted useful → higher probability
      - Primitives predicted useless → lower probability

   3. ENUMERATE WITH GUIDED GRAMMAR (Wake Phase)

      TopDownEnumerator(guided_grammar, ...)

      Priority queue uses -log_prob as cost:
      - High-probability programs → low cost → explored first
      - Low-probability programs → high cost → explored later
```

The Problem with PyPy Workers

PyPy provides 3-6x speedup but **cannot use PyTorch** (no CUDA/MPS, complex compilation). This creates a dilemma:

```
Main Process (CPython)           PyPy Worker
  PyTorch recognition model       No PyTorch available
  Can call predict_grammar()      Cannot run neural model
  But doesn't enumerate here      Must rebuild grammar from scratch
```

Solution in `run_factorial_experiment.py`: 1. Call `predict_grammar_weights()` in main process 2. Extract log probabilities as Python dict: `{prim_name: log_prob}` 3. Serialize dict to worker

via JSON 4. Worker reconstructs guided grammar from dict

Why overnight scripts failed to implement this: - `grammar_productions` parameter was added but never populated - Workers always get `None` and use default grammar - Recognition model trains but predictions are never used

---

Architecture Comparison Table

Feature	full_dreamcoder	overnight_optimized	overnight_cython	overnight_v3	factorial_exp
Enumerator	enumerate_simple	enumerate_simple	enumerate_simple	enumerate_simple	TopDownEnumerator
Uses Grammar Probs	☐	☐	☐	☐	☐
Neural Guidance	☐	☐	☐	☐	☐
Parallelization	Sequential	PyPy workers	PyPy workers	PyPy workers	CPython workers
Speedup	1x	3-6x	3-6x	3-6x	1x
Curriculum	☐	☐	☐	☐	☐
Holdout Verification	☐	☐	☐	☐	☐
Checkpoint/Resume	☐	☐	☐	☐	☐

Critical Discovery

PyPy Workers Don't Receive Neural Guidance

Impact: All overnight experiments (run_overnight_*.py) have been running **without neural guidance** during the wake phase. The recognition model trains successfully, but its predictions are never used to guide enumeration.

Evidence:

1. **enumeration_worker.py line 54:**

   ```
   grammar = build_lean_grammar()  # Always fresh default grammar
   ```

2. **enumeration_worker.py lines 57-59:**

   ```
   if grammar_productions:
       pass  # Placeholder - never executed because grammar_productions is always None
   ```

3. **run_pypy_worker() line 268:**

```
'grammar_productions': None,  # Hardcoded to None
```

4. **_run_iteration() in run_overnight_cython.py:**
    - Wake phase never calls `recognition.predict_grammar_weights()`
    - Tasks sent to workers without guidance

Consequence: The system has been doing blind iterative-deepening search, not neural-guided search. Solutions found are due to the grammar structure and enumeration order, not neural guidance.

Fix Required: Either: 1. Serialize predicted log probabilities to workers (like factorial_experiment does) 2. Or use CPython workers with TopDownEnumerator instead of PyPy

---

Recommended Next Steps

1. **For production overnight runs:** Use `run_factorial_experiment.py` architecture pattern
2. **To fix existing scripts:** Add `predicted_log_probs` serialization to PyPy workers
3. **For analysis:** Compare results with/without neural guidance to measure its effect
4. **For debugging:** The test_3iter_verification.py confirms the core components work